

目錄

1. [JPA/Hibernate 進階映射快速導覽](#) 概念:開場先說明真實世界的資料庫幾乎不會只有一張表,這門課接下來會依序教一對一、一對多、多對一、多對多這幾種資料表之間常見的關聯方式,讓你對整個章節的地圖先有個底。
2. [資料庫關聯性基礎:主鍵與外鍵](#) 概念:複習資料庫最基本的兩個角色——主鍵(每一列資料的身分證字號,保證獨一無二)和外鍵(某張表裡存著另一張表主鍵的欄位,像一條牽線把兩張表綁在一起),這是後面所有關聯映射的地基。
3. [級聯操作、抓取策略與單雙向關聯預告](#) 概念:介紹級聯(Cascading)——對父資料做的操作(存檔、刪除)要不要自動套用到子資料上;並先預告接下來會用到的及時載入/延遲載入(一次抓完 vs. 真的要用才去拿)、以及單向/雙向關聯(兩個物件是不是都能互相查到對方)這幾個之後會反覆出現的核心觀念。
4. [一對一映射:概念與資料庫層設計](#) 概念:用 Instructor(講師)與 Instructor Detail(講師的詳細資料)這組經典例子講解什麼是「一對一」關聯,並動手設計資料表結構、寫 SQL 加上外鍵約束(Foreign Key Constraint),讓資料庫自己就會擋掉不合法的關聯資料。
5. [建立 Spring Boot 專案與資料庫連線設定](#) 概念:用 Spring Initializr 產生新專案、匯入 IntelliJ,寫一個一啟動就跑的命令列測試程式,接上 application.properties 裡的資料庫帳密,並把 Hibernate 預設會狂噴的一堆 SQL 語法用日誌等級調整成看得順眼的樣子。
6. [實作 InstructorDetail 與 Instructor 的 Entity 映射](#) 概念:把 Instructor 跟 InstructorDetail 這兩個 Java 類別加上 @Entity、@Id、@Column 這些註解「貼標籤」,對應到資料庫的表跟欄位,並用 @JoinColumn 標出誰是外鍵,讓 Hibernate 知道這兩張表要透過哪個欄位串起來。
7. [建立 AppDAO、儲存與驗證一對一關聯](#) 概念:寫一個 DAO(資料存取物件,專門負責跟資料庫溝通的窗口)把 Instructor 跟它的 InstructorDetail 一起存進資料庫,靠級聯設定讓你存一次 Instructor,關聯的 Detail 資料就自動跟著寫入,再用 MySQL Workbench 打開資料庫確認真的存對了。
8. [查詢與刪除:一對一關聯的讀取和級聯刪除](#) 概念:示範怎麼依主鍵把 Instructor 連同它的 Detail 一起查回來,以及刪除 Instructor 時怎麼設定,讓 Hibernate 自動把不再需要的 Detail 資料一併清掉,不留下孤兒資料。
9. [雙向一對一映射:用 mappedBy 讓兩邊互相查得到](#) 概念:前面單向關聯只能從 Instructor 查到 Detail、查不回來;這裡改成雙向,讓兩邊都能互相查到對方,關鍵是用 mappedBy 告訴 Hibernate 「外鍵其實已經在另一邊定義過了,這邊不用重複建一個」,避免資料庫多長出一張不必要的表。
10. [雙向關聯的級聯刪除細節與斷開關聯](#) 概念:深入處理雙向關聯刪除時比較棘手的情境——怎麼調整 Cascade 類型,讓刪除 Detail 時不會連帶波及 Instructor,以及怎麼「斷開」兩個物件之間的關聯而不是直接刪掉整筆資料。
11. [JPA 一對多與多對一:資料庫設計與 mappedBy 原理](#) 概念:進入這門課份量最重的單元——一個 Instructor(講師)可以教很多 Course(課程),這叫「一對多」;反過來每個 Course 只屬於一個 Instructor 就是「多對一」,這裡先講資料庫怎麼設計、Course 表的外鍵怎麼設,以及 mappedBy 在雙向關聯裡到底扮演什麼角色。
12. [完整實作 Course 與 Instructor 的一對多雙向關聯](#) 概念:把 Course 類別寫完整(欄位、建構子、toString),用 @ManyToOne / @OneToMany 兩個註解讓 Course 跟 Instructor 雙向互相標記對方,並寫一

個 add 便利方法,讓「幫講師加一門課」的同時兩邊的關聯資料能自動維持一致,最後存進資料庫驗證關聯有沒有建對。

13. **Eager 與 Lazy Loading:抓取策略與 LazyInitializationException** 概念:解釋查一個 Instructor 時,要不要順便把底下所有課程也一次抓出來(Eager,像是套餐直接全上)還是先不抓、真的要才去拿(Lazy,像是單點現點現做),以及沒設定好 Lazy 導致查詢時噴出 LazyInitializationException 錯誤該怎麼排查解決。
14. **優化查詢:用 JOIN FETCH 一次撈出關聯資料** 概念:教你用 JPQL 的 JOIN FETCH 語法,在同一次 SQL 查詢裡就把 Instructor 跟它底下的 Course 一起撈出來,不用再靠 Lazy Loading 多發一次查詢,藉此減少資料庫來回次數、提升效能。
15. **一對多關聯的更新與刪除操作** 概念:示範怎麼更新 Instructor 跟 Course 的資料,以及刪除時遇到的資料庫「完整性約束違反」問題——因為外鍵還指著它,不能直接刪掉父資料,必須先在程式碼裡解除關聯才能安全刪除。
16. **單向一對多關聯示範:Course 與 Review** 概念:換一個新例子——一門 Course 底下有很多則 Review(評論),但這次故意只做「單向」關聯(只能從 Course 查到 Review、查不回去),讓你對照體會單向跟雙向設計上的差異跟取捨。
17. **完整實作 Review 一對多關聯練習專案** 概念:延續 Review 的例子,重新走一遍完整的開發流程——資料表設計、Entity 類別、級聯儲存、用 JOIN FETCH 查詢課程跟評論、刪除課程時連帶清掉評論,把前面學到的一對多技巧整合成一個小練習專案。
18. **多對多關聯(@ManyToMany)是什麼:連接表概念** 概念:介紹多對多關聯要靠一張額外的「連接表(Join Table)」來記錄兩邊的對應關係(例如一個學生選很多課、一門課也有很多學生),說明這張表長什麼樣子、資料怎麼存取。
19. **多對多開發起手式:Join Table 設計與反向工程** 概念:動手設計 course_student 這張連接表、用 MySQL Workbench 反向工程畫出 ER 圖確認結構正確,並開始建立 Student 這個 Entity 類別。
20. **完整實作 Course 與 Student 的多對多映射** 概念:用 @JoinTable 註解把 Course 跟 Student 兩個 Entity 透過連接表串起來,搭配 mappedBy 分清楚誰是「擁有端」誰是「非擁有端」,並寫 addStudent / addCourse 便利方法維持雙向資料一致,最後存檔驗證。
21. **多對多關聯的查詢、更新與刪除操作** 概念:示範怎麼分別從 Course 查它的學生、從 Student 查他修的課(雙向都要查得到),以及幫學生加選新課程的更新邏輯,還有刪除課程或學生時連接表資料要怎麼一併處理。
22. **AOP 概觀:從重複程式碼問題談起** 概念:用一個「幫 DAO 方法加日誌記錄、加安全性檢查」的情境,說明如果每個方法都手動加這些程式碼會多麼重複又難維護(程式碼糾結與分散),進而帶出 AOP 這種可以把這些「橫切關注點」統一抽出來管理的技術,以及它背後用代理模式(Proxy)實作的原理。
23. **Spring AOP 與 AspectJ、AOP 術語總覽** 概念:比較 Spring AOP 跟 AspectJ 這兩套 Java AOP 框架的差異與各自適用時機,並整理 AOP 的專有名詞(Aspect、Advice、Pointcut、Weaving 等)和不同 Advice 類型的總覽,方便後面對照學習。
24. **建立 AOP 示範專案與第一個 @Before Advice** 概念:動手建立第一個 AOP 練習專案,加上 Spring Boot AOP 的依賴套件,寫一個 AccountDAO 當作示範目標,再用 @Before 這個最基本的 Advice 類型,讓程式在呼叫 DAO 方法「之前」自動插入一段日誌邏輯。

25. **點切點表達式(Pointcut Expression):execution 語法與萬用字元** 概念:拆解點切點表達式(Pointcut Expression)的語法——用 execution(...) 描述要攔截哪個套件、哪個類別、哪個方法,並用星號等萬用字元放寬或收緊攔截範圍。
26. **進階點切點比對:回傳型別、參數與套件** 概念:繼續深入點切點表達式,教你怎麼依照方法的回傳型別、參數型態(甚至是任意數量的參數)、以及所在的套件來精確篩選要攔截的目標方法。
27. **點切點宣告(Pointcut Declaration):重用與組合** 概念:同一個攔截範圍常常要套用在好幾個 Advice 上,與其每次複製貼上表達式,不如用 @Pointcut 註解把它宣告成一個有名字的方法,之後所有 Advice 都可以直接引用這個名字,還能把多個點切點宣告組合起來用。
28. **在 Advice 裡存取方法參數,並攔截 Getter/Setter** 概念:示範怎麼在 Advice 裡面拿到被攔截方法實際傳進來的參數值,並印出來看,順便展示 AOP 也能攔截到看起來很普通的 Getter/Setter 方法。
29. **@AfterReturning Advice:攔截並修改方法的回傳值** 概念:@AfterReturning 是在目標方法「成功執行完畢、拿到回傳值之後」才觸發的 Advice,這裡教你怎麼在裡面存取那個回傳值,甚至進一步修改要回傳給呼叫端的資料內容。
30. **@AfterThrowing 與 @After:攔截例外與收尾動作** 概念:@AfterThrowing 專門攔截方法「拋出例外」的那個時刻,可以在裡面記錄錯誤資訊;而 @After(相當於 try-finally 裡的 finally)則是不管方法成功還是失敗都一定會執行,適合拿來做收尾清理的動作。
31. **@Around Advice:效能監控與完全掌控方法執行** 概念:@Around 是威力最強的 Advice,可以完全包住目標方法的執行過程,示範拿它來做效能監控(記錄方法跑了多久),並認識 ProceedingJoinPoint 這個用來手動觸發目標方法執行的物件。
32. **@Around 的例外處理:自己處理還是重新拋出** 概念:討論用 @Around 攔截到例外之後,是要自己在 Advice 裡把例外處理掉,還是要重新拋出(throw)讓例外繼續往上傳遞給呼叫端,兩種做法各自的影響跟怎麼選擇。
33. **實戰整合:把 AOP 日誌套進真正的 Spring MVC CRUD 專案** 概念:把前面學到的 AOP 技巧套進一個真正的 Spring MVC CRUD 專案裡,對 Controller、Service、DAO 這三層一次性加上 @Before(記錄請求進入)跟 @AfterReturning(記錄資料回傳)的日誌功能,完成一次端到端的實戰整合。

1. JPA / Hibernate 進階映射

之前學的是「基礎映射」：一個 Java Class 對一張表，簡單明瞭。但真實世界的資料庫幾乎不會只有一張表，而是一堆表互相牽扯，所以這節要進到「進階映射」——處理多張表之間的關係。這邊先把四種關係型態當作總覽掃過一遍，後面章節會逐一深挖：

- **一對一 (One-to-One)**:一個實體對應另一個實體的詳細資料,像講師 (Instructor) 有一份講師詳細資料 (Instructor Detail),概念上很像「個人檔案」。資料庫用兩張表做, `instructor` 表裡有個 `instructor_detail_id` 欄位當橋樑,指向 `instructor_detail` 表的主鍵。
- **一對多 (One-to-Many)**:一個實體可以對應多個關聯實體,例如一位講師可以開很多門課 (Course)。這裡先簡化假設一門課只由一位講師開,實際情境可能更複雜。
- **多對一 (Many-to-One)**:就是一對多的「反過來看」——很多門課指回同一位講師,是同一種關係站在子實體那端的視角而已。
- **多對多 (Many-to-Many)**:兩邊都可以有複數關聯,例如學生與課程——一門課很多學生選,一個學生也選很多課,兩邊互相交織,關係最複雜。

記住一個大原則:基礎映射是「單表對單類別」,進階映射是「多表 + 關係」,而這些關係要怎麼設計,取決於實際的資料結構長什麼樣子。

2. 資料庫關聯性基礎概念

在寫 JPA 註解之前,要先把資料庫層級的基本工具搞懂,不然後面看 `@JoinColumn`、`@OneToOne` 只會覺得是在背咒語。三個核心概念:

- **主鍵 (Primary Key)**:每張表用來唯一識別每一列資料的欄位,確保不會有兩筆一模一樣的紀錄搞混。
- **外鍵 (Foreign Key)**:某張表裡的一個欄位,它的值指向「另一張表」的主鍵,靠這個欄位把兩張表串起來。可以想成是「一張紙條,上面寫著另一張表某一列的門牌號碼」。
- **級聯操作 (Cascading)**:決定當父實體發生變化時(比如被刪除),關聯的子實體要不要跟著連動變化。

具體例子:`instructor` 表裡有個 `instructor_detail` 欄位(即 `instructor_detail_id`),這就是外鍵。假設講師 Darby 的 `instructor_detail_id` 是 100,那麼查詢時系統就會跑去 `instructor_detail` 表裡找 `id = 100` 的那筆資料,兩邊就這樣被邏輯上串起來,形成一對一關係。外鍵的值必須「對得上」目標表的主鍵,這是後面「參照完整性」的基礎。

3. 資料庫級聯操作 (Cascading)

級聯的核心定義很簡單:對父實體做的操作,自動套用到跟它相關聯的子實體上。常見兩種:

- **儲存級聯 (Save Cascade)**:儲存 `Instructor` 時,如果它關聯的 `InstructorDetail` 也是新資料,系統會順便把這筆也寫進資料庫,不用你分兩次存。
- **刪除級聯 (Delete Cascade)**:刪除 `Instructor` 時,自動把對應的 `InstructorDetail` 也刪掉。邏輯是「沒有講師了,講師的詳細資料留著也沒意義」,不刪的話就會變成資料庫裡的孤兒資料 (orphan data)。

但級聯刪除不是無腦全開的功能,一定要照著業務情境(Use Case)判斷。經典的反例是學生與課程:如果刪除一名學生就把他選的「課程」也級聯刪除掉,那就大錯特錯——課程是獨立存在的實體,不該因為一個學生離開就消失。正確做法應該是把這名學生「從課程名單移除」(解除關聯),而不是刪課程本身。所以開發者要有細粒度 (fine-grained) 的控制權,自己決定哪些關聯該級聯、哪些不該,不是為了方便就整個開下去。

這一節最後也預告了兩個後面會細講的觀念,先有個印象就好:

- **及時載入 (Eager) vs 延遲載入 (Lazy)**:抓取關聯資料時,是一次全部撈回來,還是等程式真的要用時才去資料庫多問一次。
- **單向 (Unidirectional) vs 雙向 (Bidirectional) 關聯**:單向只能從一邊查到另一邊(比如從 `Instructor` 找到 `InstructorDetail`,但反過來不行);雙向兩邊互通。沒有「唯一正確」的設計方式,一切依業務需求而定。

4. 一對一映射 (One-to-One Mapping) 的開發流程

實作一對一關聯建議照三大階段走,這個順序後面章節都會照著跑:

1. **資料庫準備 (Prep Work)**:先定義資料表結構,設好外鍵關聯。
2. **實體類別開發 (Entity Class Creation)**:先建立「被關聯」的子實體(如 `InstructorDetail`),再建立主實體(如 `Instructor`)。
3. **應用程式整合**:寫主程式把兩者串起來。

資料庫端,`instructor_detail` 表的 SQL 大致長這樣:

```
CREATE TABLE instructor_detail (  
    id INT NOT NULL AUTO_INCREMENT,  
    youtube_channel VARCHAR(255),  
    hobby VARCHAR(255),  
    PRIMARY KEY (id)  
);
```

`instructor` 表除了基本欄位(`first_name`、`last_name`、`email`)外,要多留一個 `instructor_detail_id` 欄位當「把手」。但光加欄位還不夠,兩張表在資料庫層面仍是各自獨立,要靠 `CONSTRAINT` 正式定義外鍵才算真的連上:

```
CONSTRAINT fk_instructor_detail  
    FOREIGN KEY (instructor_detail_id)  
    REFERENCES instructor_detail(id)
```

這個約束的意義叫**參照完整性 (Referential Integrity)**——白話講就是「A 引用了 B,那 B 一定要真的存在」。有了它,資料庫會主動擋掉兩種爛資料:插入一筆指向不存在 ID 的孤兒紀錄,或刪除一筆還被別人引用中的主表紀錄。

Java 端,子實體 `InstructorDetail` 先用 `@Entity + @Table(name="instructor_detail")` 對應資料表,欄位逐一映射。主實體 `Instructor` 也照做。接著關鍵的一步是把兩個原本獨立的 Java 物件串起來:在 `Instructor` 裡加一個 `InstructorDetail` 型別的屬性,標上 `@OneToOne`,再搭配 `@JoinColumn` 告訴 Hibernate 用哪個欄位當外鍵:

```
@OneToOne(cascade = CascadeType.ALL)  
private InstructorDetail instructorDetail;
```

Hibernate 之後會自動做三件事:讀 `@JoinColumn` 指定的外鍵值 → 拿這個值去關聯表查詢 → 把查到的物件組裝進記憶體中的 `Instructor` 物件裡,整個過程開發者不用手動處理。

在深入 `Cascade Type` 之前,順帶認識一下 **Hibernate 實體生命週期**——這決定 Hibernate 怎麼追蹤物件變化:

- **Transient(暫時)**:剛 `new` 出來,還沒跟資料庫扯上關係。
- **Persistent(受管理)**:已經跟 Session 綁定,Hibernate 在盯著它的變化。
- **Detached(游離)**:曾經受管理,但 Session 關掉後就斷線了,記憶體裡物件還在,但改了也不會自動同步進資料庫。
- **Removed**:對受管理物件呼叫 `remove` 之後,等 `commit` 才真的從資料庫刪掉。

四個對應操作方法:`persist`(讓新實體變 Managed)、`merge`(把 Detached 物件重新接回 Session)、`remove`(標記刪除)、`refresh`(強制用資料庫最新資料覆蓋記憶體裡的舊值,避免 Stale Data)。不用死背所有轉換細節,重點抓住「物件跟 Session 的關聯性」跟「記憶體與資料庫何時同步」這兩個大方向就好。

最後是 **Cascade Type** 的完整清單,不是全開全關的開關,而是可以挑著配置:

Cascade Type	說明
PERSIST	主實體被存,關聯實體也跟著存
REMOVE	主實體被刪,關聯實體也跟著刪
REFRESH	主實體同步,關聯實體也跟著從資料庫更新
DETACH	主實體游離,關聯實體也跟著游離
MERGE	主實體合併,關聯實體也跟著合併
ALL	以上全部

預設是完全不級聯,不寫 `cascade` 屬性,Hibernate 什麼都不會自動連動。要精細控制的話可以用逗號分隔的清單,例如只要 PERSIST 跟 REMOVE:

```
@OneToOne(cascade = {CascadeType.PERSIST, CascadeType.REMOVE})
private InstructorDetail instructorDetail;
```

5. 建立 Spring Boot 命令列應用程式 (Command Line App)

這節目標是搭一個專門拿來測試 JPA/Hibernate 的命令列小工具,架構沿用之前學過的 **DAO 模式**:Main App 負責流程,AppDAO 負責跟資料庫講話。

先定義 DAO 介面,只放一個 `save`:

```
public interface AppDAO {
    public void save(Instructor theInstructor);
}
```

實作類別 `AppDAOImpl` 裡放一個 `EntityManager`,用建構子注入取得(比 field injection 更推薦的做法),`save` 方法就是一行 `entityManager.persist(theInstructor)`。因為 `Instructor` 上配置了 `CascadeType.ALL`,呼叫一次 `persist` 就會連 `InstructorDetail` 一起存進去,不用分開存兩次。

主程式 (`CommandLineRunner`) 這邊,重點是先把兩個物件建立好、用 `setter` 關聯起來,才能呼叫 `save`:

```
tempInstructor.setInstructorDetail(tempInstructorDetail);
appDAO.save(tempInstructor);
```

漏掉 `setInstructorDetail` 這一步,Hibernate 完全不知道兩個物件有關係,級聯也不會發生——這是最容易漏掉的地雷。

這章也花了不少篇幅在環境搭建的細節,可以當作 checklist:

- 用 start.spring.io 建專案,記得勾 **MySQL Driver** 跟 **Spring Data JPA** 這兩個依賴。
- `application.properties` 要設資料庫連線三件套:

```
spring.datasource.url=jdbc:mysql://localhost:3306/hb-01-one-to-one-uni
spring.datasource.username=springstudent
spring.datasource.password=springstudent
```

- 因為是獨立命令列工具,不想看到一堆啟動雜訊,可以關掉 Banner 跟調降日誌層級:

```
spring.main.banner-mode=off
logging.level.root=warn
```

調成 `warn` 不代表看不到問題——警告跟錯誤還是會照常顯示,只是把正常的背景日誌濾掉,讓輸出乾淨到只剩你自己 `print` 的東西。

另外補一個概念:在 `Instructor` ↔ `InstructorDetail` 這組單向關聯裡,擁有方 (**Owning Side**) 是 `Instructor`,因為外鍵實際上放在 `instructor` 表裡。誰是擁有方,誰的表就負責存外鍵——這個角色分配之後在雙向映射時會很關鍵。

6. 一對一映射開發流程：建立 `InstructorDetail` 實體類別

這節是把第 4 節講的原則,一步步落實成完整程式碼。整體步驟回顧:1. 準備工作 2. 建立 `InstructorDetail` 類別 3. 建立 `Instructor` 類別 4. 建立主應用程式。先在 `com.luv2code.cruddemo.entity` 底下建 `entity` 套件統一放實體類別。

`InstructorDetail` 的完整寫法:

```
@Entity
@Table(name="instructor_detail")
public class InstructorDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="id")
    private int id;

    @Column(name="youtube_channel")
    private String youtubeChannel;

    @Column(name="hobby")
    private String hobby;

    // 建構子(只帶 youtubeChannel、hobby,不含 id)
    // getter/setter、toString() 用 IDE 產生
}
```

主鍵用 `@Id + @GeneratedValue(strategy = GenerationType.IDENTITY)`,意思是讓 MySQL 自己用 `AUTO_INCREMENT` 處理 `id`,程式不用管。

Instructor 類別作法一樣,先建基本欄位(`id`、`firstName`、`lastName`、`email`),用 `@Column` 對應到 `first_name`、`last_name` 等資料庫欄位名。接著補上跟 `InstructorDetail` 的關聯:

```
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name="instructor_detail_id")
private InstructorDetail instructorDetail;
```

`@JoinColumn` 就是 Hibernate 跟資料庫之間的「掛鉤」——它告訴 Hibernate:「請用這個欄位去做 join,完成一對一映射」,而這個欄位名稱必須跟 SQL 腳本裡定義的外鍵約束一致,兩邊對不上就會出錯。

建構子生成時有個小技巧值得注意:用 IDE 產生建構子時,故意排除 `id` 跟 `instructorDetail` 這兩個欄位——`id` 因為是資料庫自動生成不用手動傳,`instructorDetail` 則是希望物件建好後再用 setter 手動關聯(或靠級聯處理),不要在建構子裡強制綁定。所以最終建構子長這樣:

```
public Instructor(String firstName, String lastName, String email) {
    this.firstName = firstName;
    this.lastName = lastName;
    this.email = email;
}
```

`toString()` 記得把 `instructorDetail` 也印進去,方便之後除錯時一次看清楚整個關聯物件的內容。

另外這節花了不少篇幅在 MySQL Workbench 的操作(建 Schema、執行腳本、Reverse Engineer 產生 ER 圖),重點只有一個容易忽略的觀念:**Workbench 的 ER 圖有時會誤判關聯基數(比如把一對一顯示成別的類型),但這只是視覺上的顯示問題(Cosmetic),不影響資料庫實際的結構或約束**。真正決定關聯行為的,是 Java 程式碼裡的 `@OneToOne` 設定,不是圖表長什麼樣子。級聯行為本身,課程也建議不要寫在 SQL 腳本層級(例如 `ON DELETE CASCADE`),而是交給 Hibernate 在應用程式層管理,彈性比較大。

7. 建立 AppDAO 介面

AppDAO 介面先加上 `save`:

```
package com.luv2code.crulldemo.dao;

public interface AppDAO {
    void save(Instructor theInstructor);
}
```

AppDAOImpl 實作類別的重點三件事:

1. 建構子注入 **EntityManager** (`@Autowired` 加不加都行,但加了可讀性較好)。
2. `save` 方法要標 `@Transactional`,因為這是寫入資料庫的操作,得包在交易裡。
3. 呼叫 `entityManager.persist(theInstructor)`,靠先前設定的 `CascadeType.ALL` 自動連帶存 `InstructorDetail`。


```
@Repository
public class AppDAOImpl implements AppDAO {
    private EntityManager entityManager;

    @Autowired
    public AppDAOImpl(EntityManager entityManager) {
        this.entityManager = entityManager;
    }

    @Override
    @Transactional
    public void save(Instructor theInstructor) {
        entityManager.persist(theInstructor);
    }
}
```

踩雷提醒:一開始若忘記在 `AppDAOImpl` 上加 `@Repository`, 啟動時會直接爆錯——Spring 找不到 `AppDAO` 型別的 Bean 可以注入到 `CommandLineRunner` 裡, 因為 Component Scan 沒把這個類別當成受管理的元件。加上 `@Repository` 就解決。

主程式端把邏輯包進 `createInstructor` 方法, 流程是: new 兩個物件 → `setInstructorDetail` 建立關聯 → 呼叫 `appDAO.save`:

```
private void createInstructor(AppDAO appDAO) {
    Instructor tempInstructor = new Instructor("Chad", "Darby",
"darby@luv2code.com");
    InstructorDetail tempInstructorDetail =
        new InstructorDetail("http://www.luv2code.com/youtube", "luv 2
code!!!");

    tempInstructor.setInstructorDetail(tempInstructorDetail);

    // NOTE: 因為 CascadeType.ALL, 這裡也會連帶存 detail
    appDAO.save(tempInstructor);
}
```

打開 SQL 日誌(`logging.level.org.hibernate.SQL=trace`、`...jdbc.bind=trace`)後可以觀察到一個容易忽略但很重要的細節:**Hibernate** 儲存時是先 insert `instructor_detail`, 再 insert `instructor`——因為 `instructor` 要靠外鍵欄位記住 `instructor_detail` 的 id, 所以子表(關聯方)必須先寫入才能拿到 id 回填。

查詢的部分, `AppDAO` 加上 `findInstructorById`:

```
@Override
public Instructor findById(int theId) {
    return entityManager.find(Instructor.class, theId);
}
```

重要觀念:@OneToOne 預設的抓取策略 (Fetch Type) 是 **EAGER(立即載入)**,所以查 **Instructor** 時,關聯的 **InstructorDetail** 也會自動一起撈出來,不用額外查詢。測試方法印出 **tempInstructor.getInstructorDetail()** 就能驗證這點。之後換不同 id 測試(如 Madhu Patel 那筆),結果也都符合預期,證明一對一映射跟級聯儲存都運作正常。

8. JPA / Hibernate 一對一：刪除實體

刪除一個帶有一對一關聯的實體分兩步:先用 **entityManager.find()** 把物件撈出來,再用 **entityManager.remove()** 傳進去刪除。跟儲存邏輯一樣,因為配置了 **CascadeType.ALL**,刪主實體時關聯實體也會自動被刪掉,不用分開處理兩次:

```
@Override
@Transactional
public void deleteInstructorById(int theId) {
    // retrieve the instructor
    Instructor tempInstructor = entityManager.find(Instructor.class,
theId);

    // delete the instructor
    entityManager.remove(tempInstructor);
}
```

AppDAO 介面對應加上 **void deleteInstructorById(int theId);**,主程式測試方法一樣簡單:指定一個已存在的 id,呼叫 DAO 刪除,印出訊息確認。

觀察 SQL 日誌會發現一個跟儲存時相反的細節:儲存時是先插 **instructor_detail** 再插 **instructor**;但刪除時卻是先刪 **instructor**,再刪 **instructor_detail**:

```
delete from instructor where id=?
delete from instructor_detail where id=?
```

這個順序差異很容易搞混,記憶點是:插入要先滿足外鍵值的來源(子表先建立才有 id 可用),但刪除時反過來——先斷開持有外鍵的那一方,再清掉被指向的那一方。最後用 MySQL Workbench 查兩張表都確認對應 id 的紀錄已經一起消失,驗證級聯刪除確實生效。

9. 單向映射 (Uni-directional Mapping) 的限制

目前為止做的都是單向映射,關係只能單向流動:**Instructor** → **InstructorDetail**。也就是說,拿到一個 **Instructor** 物件可以透過它取得對應的 **InstructorDetail**,但反過來——如果你手上先拿到的是 **InstructorDetail** 物件,完全沒辦法從它身上反查回對應的 **Instructor**。這就是單向映射天生的限制。

新的使用情境:假設需求變成「載入一個 `InstructorDetail`,同時要能直接拿到與它關聯的 `Instructor`」,單向映射就做不到,得把關係升級成雙向映射 (**Bidirectional Mapping**),讓兩邊都能互相導航。好消息是:升級成雙向完全不用改資料庫結構,`instructor` 跟 `instructor_detail` 兩張表原封不動繼續用,所有改動都只發生在 Java 程式碼層。

實作步驟就兩步:

1. 更新 `InstructorDetail` 類別:加一個引用 `Instructor` 的新欄位,配上 getter/setter,再標上 `@OneToOne`。
2. 更新主應用程式,寫測試驗證雙向導航真的通了。

`InstructorDetail` 新增的欄位長這樣:

```
@Entity
@Table(name="instructor_detail")
public class InstructorDetail {

    @OneToOne(mappedBy="instructorDetail", cascade=CascadeType.ALL)
    private Instructor instructor;

    public Instructor getInstructor() { return instructor; }
    public void setInstructor(Instructor instructor) { this.instructor =
instructor; }
}
```

這裡的關鍵是 `mappedBy` 屬性,很多人第一次看會搞混,拆解一下它的運作邏輯:

輯:`mappedBy="instructorDetail"` 是在告訴 Hibernate——「這個關聯的物理映射(也就是外鍵)已經在 `Instructor` 類別的 `instructorDetail` 欄位定義過了,你直接去那邊看 `@JoinColumn` 設定就好,我這邊不重複定義外鍵」。所以整組關聯裡:

- 擁有方 (Owning Side) = `Instructor`:用 `@OneToOne + @JoinColumn` 定義實際的物理映射,外鍵欄位 (`instructor_detail_id`) 真正存在 `instructor` 這張表裡。
- 反向端 (Inverse Side) = `InstructorDetail`:用 `@OneToOne(mappedBy="instructorDetail")`,單純鏡像擁有方的狀態,自己不持有任何外鍵欄位。

角色	實體類別	關鍵註解	資料庫行為
擁有方	<code>Instructor</code>	<code>@OneToOne + @JoinColumn</code>	在自己表中建立外鍵欄位
反向方	<code>InstructorDetail</code>	<code>@OneToOne(mappedBy="...")</code>	不持有外鍵,只是鏡像擁有方

反向端一樣可以加 `cascade=CascadeType.ALL`,效果是「如果從 `InstructorDetail` 這端執行刪除,也會連帶刪掉關聯的 `Instructor`」——這點值得留意,雙向級聯代表兩個方向都可能觸發連鎖刪除,不是只有擁有方才能發動級聯。

DAO 與主程式的更新跟前面套路一樣:`AppDAO` 加一個 `findInstructorDetailById`,實作用 `entityManager.find(InstructorDetail.class, theId)`。因為 `@OneToOne` 預設 `EAGER`,查出來的 `InstructorDetail` 物件會自動連帶取得關聯的 `Instructor`。主程式測試方法:

```
private void findInstructorDetail(AppDAO appDAO) {
    int theId = 2;
    InstructorDetail tempDetail = appDAO.findInstructorDetailById(theId);

    System.out.println("Instructor detail: " + tempDetail);
    System.out.println("Associated instructor: " +
tempDetail.getInstructor());
}
```

驗證時打開 SQL 日誌可以看到,Hibernate 查詢 `instructor_detail` 時會自動夾帶一個 `LEFT JOIN instructor`,把兩邊資料一次撈齊:

```
select i1_0.id, i1_0.hobby, i2_0.id, i2_0.email, i2_0.first_name,
i2_0.last_name, i2_0.youtube_channel
from instructor_detail i1_0
left join instructor i2_0 on i1_0.instructor_id = i2_0.id
where i1_0.id=?
```

控制台印出的 `tempInstructorDetail` 與 `associated instructor` 內容跟資料庫紀錄完全對得上,證明雙向導航真的成立了。至此開發者手上握有完整的雙向自由:路徑 A——從 `Instructor` 出發拿到 `InstructorDetail`;路徑 B——從 `InstructorDetail` 出發拿到 `Instructor`,單向映射「查不回去」的限制正式被打破。

10. 雙向一對一關聯 (Bidirectional One-to-One) 的特性

雙向關聯就像是兩個人互相有對方的電話號碼——不只 `Instructor` 可以找到 `InstructorDetail`, `InstructorDetail` 也能反過來查到自己屬於哪個 `Instructor`。這一節主要示範「刪除從屬物件時,該不該連帶刪掉主物件」的問題。

一開始 `InstructorDetail` 上設定 `cascade = CascadeType.ALL`,結果測試刪除 `InstructorDetail` 時,Hibernate 產生了兩條 SQL:

```
delete from instructor_detail where id=?
delete from instructor where id=?
```

這代表只要刪詳細資訊,講師本人也會被一起「陪葬」——這通常不是我們要的行為。解法是把 `CascadeType.ALL` 換成精細控制,只留下不含 `REMOVE` 的組合:

```
@OneToOne(mappedBy = "instructor", cascade = {
    CascadeType.DETACH, CascadeType.MERGE,
    CascadeType.PERSIST, CascadeType.REFRESH})
private Instructor instructor;
```

但光改 cascade 還不夠。因為 `Instructor` 那端才是外鍵擁有者(有 `instructor_detail_id` 欄位),如果刪除 `InstructorDetail` 前沒有先斷開雙向的記憶體引用,資料庫還是會抱怨外鍵有人在用。所以 DAO 的刪除方法要先手動「分手」,再刪除:

```
@Transactional
public void deleteInstructorDetailById(int theId) {
    InstructorDetail tempInstructorDetail =
        entityManager.find(InstructorDetail.class, theId);

    // 先斷開雙向連結,把 instructor 那端的外鍵設為 null
    tempInstructorDetail.getInstructor().setInstructorDetail(null);

    entityManager.remove(tempInstructorDetail);
}
```

實際執行後 Hibernate 會先 `UPDATE instructor SET instructor_detail_id=null`,再 `DELETE FROM instructor_detail`。驗證結果是:`instructor_detail` 那筆記錄消失了,但 `instructor` 的記錄還在(只是外鍵欄位變成 null)。這就是「精細控制 cascade + 手動斷開雙向引用」兩個動作合力達成「只刪子物件、保留父物件」的效果。

11. JPA / Hibernate 一對多與多對一映射

這節開始進入一對多(One-to-Many)/多對一(Many-to-One)的世界,場景換成「一位講師可以教很多門課」。兩個註解剛好是一體兩面:

- `Instructor` 端用 `@OneToMany(mappedBy="instructor")` 看向一堆 `Course`
- `Course` 端用 `@ManyToOne + @JoinColumn(name="instructor_id")` 指回單一講師

業務需求很明確:刪講師不要牽連課程,刪課程也不要牽連講師,所以兩邊的 cascade 都要排除 `REMOVE`,只留 `PERSIST`、`MERGE`、`DETACH`、`REFRESH`。

資料庫層先建好 `course` 表,`instructor_id` 當外鍵指向 `instructor.id`,`title` 加 `UNIQUE` 避免重複課名:

```
CREATE TABLE course (
    id int(11) NOT NULL AUTO_INCREMENT,
    title varchar(128) DEFAULT NULL,
    instructor_id int(11) DEFAULT NULL,
    PRIMARY KEY (id),
    UNIQUE KEY TITLE_UNIQUE (title),
    CONSTRAINT FK_INSTRUCTOR FOREIGN KEY (instructor_id) REFERENCES
instructor (id)
);
```

`Instructor` 這端新增集合欄位,並設好對稱的 cascade:


```
@OneToMany(mappedBy="instructor",
            cascade={CascadeType.PERSIST, CascadeType.MERGE,
                    CascadeType.DETACH, CascadeType.REFRESH})
private List<Course> courses;
```

`mappedBy="instructor"` 的意思是:「這段關係的實際控制權在 `Course` 類別裡叫 `instructor` 的那個屬性身上」——白話講就是 `Instructor` 這邊只是掛個名(反向方 / inverse side),真正外鍵長在 `Course` 表上(擁有方 / owning side)。

雙向關聯光靠 setter 各設一半容易漏東漏西,所以通常會在 `Instructor` 加一個「一次到位」的便利方法,同時更新集合跟對方物件,確保兩邊的引用永遠同步:

```
public void addCourse(Course tempCourse) {
    if (courses == null) {
        courses = new ArrayList<>();
    }
    courses.add(tempCourse);
    tempCourse.setInstructor(this); // 讓 Course 那端也認得 instructor
}
```

整個一對多的開發流程可以濃縮成四步驟:①定資料表 → ②建 `Course` 類別 → ③改 `Instructor` 類別 → ④寫主程式驗證。用 MySQL Workbench 的反向工程功能把 schema 拉成 EER 圖,可以直觀確認 `instructor` 與 `course` 之間確實是一對多關係。

12. 建立 `Course` 實體類別

延續上一節的規劃,這節專門把 `Course` 類別從零蓋起來。開發順序是:先開一個新專案(從既有的一對一專案複製過來,避免動到舊程式),再照「欄位 → 建構子 → getter/setter → toString → 加註解」的順序完成類別。

欄位對應資料庫三個欄位:`id`(主鍵)、`title`(課程標題)、`instructor`(關聯物件,對應外鍵 `instructor_id`)。建構子刻意只收 `title` 一個參數——`id` 交給資料庫自動編號,`instructor` 則留給之後用 setter 或 `addCourse()` 去指定,不硬塞進建構子裡。

`toString()` 也有小技巧:自動產生時故意不勾選 `instructor` 欄位,理由是避免印出物件時觸發不必要的關聯查詢,甚至造成雙向物件互相呼叫 `toString()` 而無限循環。

欄位註解部分,`id` 用資料庫自動遞增策略,`title` 一般欄位映射,`instructor` 則是這節的重點——多對一關聯搭配細粒度 cascade(排除 `REMOVE`):

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id")
private int id;

@Column(name = "title")
private String title;
```

```
@ManyToOne(cascade = {CascadeType.PERSIST, CascadeType.MERGE,
                      CascadeType.DETACH, CascadeType.REFRESH})
@JoinColumn(name = "instructor_id")
private Instructor instructor;
```

完成 `Course` 之後回頭補 `Instructor` 端的 `@OneToMany(mappedBy="instructor", cascade={...})`, 並產生 `courses` 的 getter/setter, 再加上前一節提到的 `addCourse()` 便利方法。

主程式驗證時記得先把 `application.properties` 的 `spring.datasource.url` 指到新的一對多 schema, 再寫一個 `createInstructorWithCourses` 方法: 建立講師 → 建立 `InstructorDetail` 並關聯 → 建立多個 `Course` 並用 `addCourse()` 掛上去 → 最後只呼叫一次 `appDAO.save(tempInstructor)`。因為設了 `CascadeType.PERSIST`, 存講師的同時 Hibernate 會自動把底下的課程也一起存進去, 不用逐一呼叫 `save`。實際觀察到的 SQL 執行順序是:

```
insert into instructor_detail (...) values (...)
insert into instructor (...) values (...)
insert into course (instructor_id, title) values (?, ?)
insert into course (instructor_id, title) values (?, ?)
```

先存父層(detail、instructor), 再依序存子層(每筆 course), 證明級聯儲存確實按依賴順序執行。另外提到一個小知識點: `course` 表的 id 不是從 1 開始, 是因為建表腳本裡 `AUTO_INCREMENT=10`——這只是 SQL 腳本自己設定的起始值, 跟程式邏輯無關。

13. Fetch Types: Eager vs Lazy Loading

抓取策略(Fetch Type)決定 Hibernate 「什麼時候」把關聯資料一起拿回來, 是效能上很關鍵的一個開關。

- **Eager Loading(急迫載入)**: 查主實體的同時, 把所有關聯資料一次抓齊。
- **Lazy Loading(延遲載入)**: 先只拿主實體, 等程式碼真的呼叫到關聯欄位(例如 `getCourses()`)時, 才臨時再發一次查詢去拿。

打個比方: Eager 像是叫外送時把整間餐廳的菜單全部點一輪送過來, 不管你吃不吃得完; Lazy 則是先上主餐, 你想吃甜點才另外加點。當一個 `Course` 底下掛了上萬名學生時, 若用 Eager 抓, 每次查課程都要把上萬筆學生資料撈出來——這就是效能噩夢。業界慣例是預設優先用 **Lazy**, 只有明確需要的時候才切成 Eager, 例如:

- 講師列表頁(Master View)只需要姓名 Email → 用 Lazy, 別載入課程明細
- 講師詳情頁(Detail View)需要完整課程清單 → 這時用 Eager 或主動抓取才合理

各種映射關係的預設 fetch type 不一樣, 務必背起來, 否則很容易在不知情狀況下觸發效能問題:

映射類型	預設 Fetch Type
@OneToOne	EAGER
@ManyToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

可以用 `fetch` 屬性覆寫預設值,例如把通常是 EAGER 的 `@ManyToOne` 手動改成 LAZY:

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "instructor_id")
private Instructor instructor;
```

Lazy Loading 有個重要限制:它依賴一個「還開著的」Hibernate Session。如果 DAO 方法執行完、Session 已經關閉,之後才去呼叫 `getCourses()`,Hibernate 想發查詢卻發現沒有可用連線,就會丟出:

```
org.hibernate.LazyInitializationException: could not initialize a
collection when the session had been closed
```

這節示範了踩雷的完整過程:一開始 `courses` 是 LAZY, `findInstructorById()` 拿到講師後,Session 就關了,接著呼叫 `tempInstructor.getCourses()` 直接爆炸。最快的解法是把 `fetch` 改回 `FetchType.EAGER`,查講師時順便把課程也撈出來,問題就消失了——但這只是暫時方案,犧牲了 Lazy 帶來的效能優勢。

接著嘗試另一條路:保持 `fetch=LAZY` (其實這也是 `@OneToMany` 的預設值,寫出來只是為了可讀性),另外寫一個獨立方法 `findCoursesByInstructorId`,用 JPQL 主動查課程:

```
@Override
public List<Course> findCoursesByInstructorId(int theId) {
    TypedQuery<Course> query = entityManager.createQuery(
        "from Course where instructor.id = :data", Course.class);
    query.setParameter("data", theId);
    return query.getResultList();
}
```

但第一次測試又踩到同一個 `LazyInitializationException`——原因是查出 `courses` 列表後,忘記把它塞回講師物件(`tempInstructor.setCourses(courses)`),所以講師物件裡的 `courses` 欄位仍是未初始化狀態。補上這一步手動關聯後,才順利在同一個 Session 範圍內完整驗證雙向關係。這個踩坑經驗也直接鋪陳出下一節「用 JOIN FETCH 一次查完」的動機。

14. 優化查詢：在單一查詢中取得講師與課程

上一節「先查講師、再查課程、再手動關聯」的做法能用,但要付出兩次資料庫往返的代價。這節的目標是:保持關聯設定為 LAZY (不去動實體類別的 fetch type),但透過 JPQL 的 `JOIN FETCH` 語法,在一次查詢裡把講師和課程一起撈出來。

```
@Override
public Instructor findInstructorByIdJoinFetch(int theId) {
    TypedQuery<Instructor> query = entityManager.createQuery(
        "select i from Instructor i " +
        "join fetch i.courses " +
        "where i.id = :data", Instructor.class);
```

```
query.setParameter("data", theId);
return query.getSingleResult();
}
```

JOIN FETCH 的效果很像「臨時把這次查詢的抓取策略切成 EAGER」,但只影響這一次查詢,實體類別本身的預設 **LAZY** 完全不受影響。這帶來的好處是**查詢策略可以隨業務場景自由切換**:只要講師基本資料就呼叫原本的 **findInstructorById**(省效能),需要連課程一起看就呼叫 **findInstructorByIdJoinFetch**(省往返次數),不用為了某個特例把整個實體改成 EAGER 拖累其他所有查詢。

底層產生的 SQL 大致長這樣,一次 **JOIN** 就把講師與課程橋接起來:

```
select i.id, c.instructor_id, c.id, c.title, ...
from instructor i
join course c on i.id = c.instructor_id
```

這技巧還能再疊加。如果一次查詢裡同時需要 **courses** 和 **instructorDetail** 兩個關聯,可以在 JPQL 裡串接多個 **JOIN FETCH**,把原本要分開查的兩次往返合併成一次「大查詢」:

```
TypedQuery<Instructor> query = entityManager.createQuery(
    "select i from Instructor i " +
    "JOIN FETCH i.courses " +
    "JOIN FETCH i.instructorDetail " +
    "where i.id = :data", Instructor.class);
```

這就是解決 N+1 查詢問題的標準手法:與其讓每個關聯各自觸發一次額外查詢(N+1),不如一次 JPQL 把所有需要的關聯都用 **JOIN FETCH** 串起來,用一次資料庫往返換取完整資料。

15. 更新 Instructor 的流程

更新一筆實體資料的邏輯,不管是講師還是課程,套路都一樣三步驟:**找到(find)→ 改屬性(setter)→ 存回去(update)**。DAO 層的核心就是靠 **entityManager.merge()** 把記憶體裡修改過的物件狀態同步回資料庫:

```
@Override
@Transactional
public void update(Instructor tempInstructor) {
    entityManager.merge(tempInstructor);
}
```

merge() 一定要包在 **@Transactional** 裡,因為這是會寫入資料庫的動作。主程式的測試寫法也是同一個公式:

```
private void updateInstructor(AppDAO appDAO) {
    int theId = 1;
    Instructor tempInstructor = appDAO.findInstructorById(theId);
```

```
tempInstructor.setLastName("TESTER");
appDAO.update(tempInstructor);
}
```

先用 `findInstructorById` 撈出現有物件,改個姓氏,再丟給 `update()`。用 MySQL Workbench 重新整理資料表,就能看到 `last_name` 真的從舊值變成 `TESTER`,確認 `merge()` 有把變更寫進資料庫。

`Course` 的更新是一模一樣的套路,只是換個實體、換個方法:`findCourseById` 找出課程,改標題,再 `update()`。DAO 對應方法也是「find + merge」兩招走天下:

```
@Override
public Course findCourseById(int theId) {
    return entityManager.find(Course.class, theId);
}

@Override
@Transactional
public void update(Course tempCourse) {
    entityManager.merge(tempCourse);
}
```

刪除的部分,課程和講師的難度差很多。刪 `Course` 很單純,因為它是「多」的那一端,沒有別人靠它撐腰,直接 find 完 remove 就結束:

```
@Override
@Transactional
public void deleteCourseById(int theId) {
    Course tempCourse = entityManager.find(Course.class, theId);
    entityManager.remove(tempCourse);
}
```

刪 `Instructor` 就麻煩多了,因為它是「一」的那端,底下可能還掛著好幾門課程,而 `cascade` 又刻意排除了 `REMOVE`(前面幾節就是為了避免刪講師連帶刪課程)。如果沒處理好直接刪,資料庫會用外鍵約束擋下來,丟出類似這樣的錯誤:

```
Caused by: java.sql.SQLIntegrityConstraintViolationException:
Cannot delete or update a parent row: a foreign key constraint fails
(`course`, CONSTRAINT `FK_INSTRUCTOR` FOREIGN KEY (`instructor_id`)
REFERENCES `instructor` (`id`))
```

道理很直觀:課程表裡還有人的 `instructor_id` 指著這位講師,資料庫當然不准你把這個「被引用中」的講師刪掉。解法是刪除前先幫每一門課「解除關係」,把它們的 `instructor` 欄位設成 `null`,確認沒有課程還指著這位講師了,才真正執行刪除:

```

@Override
@Transactional
public void deleteInstructorById(int theId) {
    Instructor tempInstructor = entityManager.find(Instructor.class,
theId);

    List<Course> courses = tempInstructor.getCourses();
    for (Course tempCourse : courses) {
        tempCourse.setInstructor(null);    // 逐一解除關聯
    }

    entityManager.remove(tempInstructor);
}

```

實際驗證結果符合預期:講師那筆記錄從 `instructor` 表消失,但原本關聯的課程(如 id 10、11)依然留在 `course` 表裡,只是它們的 `instructor_id` 都變成了 `NULL`——課程沒有跟著陪葬,只是暫時變成「沒有老師教」的狀態。這正好呼應這一路以來反覆強調的原則:一對多關係裡,子層級的資料不該因為父層級被刪除就整批消失,該由開發者自己決定「解除關聯」還是「一併刪除」。

16. 單向一對多關聯 (@OneToMany: Uni-Directional)

這節在講「單向」的一對多關聯:一個 `Course` 可以有很多則 `Review`(評論),但反過來 `Review` 不需要知道自己屬於哪個 `Course`——關係是單行道,只從 `Course` 指向 `Review`。

- **實務需求:**刪除課程時,必須連帶刪除該課程底下所有評論(級聯刪除),因為沒有課程的評論在系統裡沒有意義,就像刪掉一支 YouTube 影片,底下的留言也該一併消失。
- **資料表設計:**`review` 表用 `course_id` 當外鍵指回 `course` 表:

```

CREATE TABLE `review` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `comment` varchar(256) DEFAULT NULL,
  `course_id` int(11) DEFAULT NULL,
  KEY `FK_COURSE_ID_idx` (`course_id`),
  CONSTRAINT `FK_COURSE`
    FOREIGN KEY (`course_id`)
    REFERENCES `course` (`id`)
);

```

- **開發流程(4 步驟):**1. 定義資料表 → 2. 建立 `Review` 類別 → 3. 更新 `Course` 類別(加關聯) → 4. 寫主程式整合測試。
- **關鍵映射寫法預告**(細節會在下一節實作):

```

@OneToMany(fetch = FetchType.LAZY, cascade = CascadeType.ALL)
@JoinColumn(name = "course_id")
private List<Review> reviews;

```


- `@JoinColumn(name="course_id")`:告訴 Hibernate 去 `review` 表找 `course_id` 欄位,藉此判斷哪些評論屬於目前這個課程。外鍵是放在「多」的那一方(`review` 表),不是放在 `course` 表。
- `cascade = CascadeType.ALL`:操作(存、刪、更新)會從 `Course` 傳遞到 `reviews`,符合「砍課程就砍評論」的需求。
- `fetch = FetchType.LAZY`:評論資料延遲載入,真正用到時才去資料庫抓,避免每次讀課程都順便撈一大包評論拖累效能。
- 擁有方 vs 反向方:單向關聯只有擁有方(`Course`,用 `@OneToMany` + `@JoinColumn`),沒有反向方——「反向方」的概念只存在雙向關聯裡。
- 前置作業:用 MySQL Workbench 開啟 `hb-04-one-to-many-uni/create-db.sql` 建新 schema、更新 `application.properties` 的 `spring.datasource.url` 指向新 schema、清空 `CommandLineRunner` 裡舊的測試程式碼,準備開始寫新功能。

17. 建立 `Review` 實體類別

這節開始動手實作:先蓋好 `Review` 這個 entity,再回頭讓 `Course` 認得它。

- 建立 `Review` 類別,標準流程:定義欄位 → 建構子 → getter/setter → toString → 註解欄位:

```
@Entity
@Table(name="review")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "comment")
    private String comment;

    public Review() { }

    public Review(String comment) {
        this.comment = comment;
    }
    // getter/setter, toString 略
}
```

- 建構子刻意只帶 `comment` 參數,因為 `id` 交給資料庫自動產生、`course_id` 之後由關聯設定處理,不用手動塞。
- 更新 `Course`,加入評論集合並配上 `@OneToMany` + `@JoinColumn` + `cascade` + `lazy`(完整版):

```
@OneToMany(fetch = FetchType.LAZY, cascade = CascadeType.ALL)
@JoinColumn(name = "course_id")
private List<Review> reviews;

// 便利方法,避免 NullPointerException
public void addReview(Review theReview) {
    if (reviews == null) {
```

```

        reviews = new ArrayList<>();
    }
    reviews.add(theReview);
}

```

- **存檔(Save):**AppDAO 新增 `save(Course theCourse)`,實作用 `entityManager.persist()`,記得加 `@Transactional`(因為會修改資料庫)。因為 `cascade=ALL`,存一次 `Course` 就會連帶把它底下所有 `Review` 一起存進去,不用逐筆手動 `persist`。
- **測試建立:**

```

Course tempCourse = new Course("Pacman - How To Score One Million Points");
tempCourse.addReview(new Review("Great course ... loved it!"));
tempCourse.addReview(new Review("Cool course, job well done."));
tempCourse.addReview(new Review("What a dumb course, you are an idiot!"));
appDAO.save(tempCourse);

```

存完後在 MySQL Workbench 查 `review` 表,三筆評論的 `course_id` 都是 10,證實級聯儲存成功。

- **查詢課程連同評論:**因為是 LAZY 載入,直接讀不到 `reviews`,要用 JPQL 的 `JOIN FETCH` 一次撈出來,避免另外觸發延遲載入查詢:

```

TypedQuery<Course> query = entityManager.createQuery(
    " select c from Course c JOIN FETCH c.reviews where c.id = :data",
    Course.class);
query.setParameter("data", theId);
Course course = query.getSingleResult();

```

小提醒:JPQL 字串開頭的引號前記得留一個空格,不然會跟前面東西黏在一起出錯。

- **刪除課程連帶刪評論:**呼叫 `appDAO.deleteCourseById(theId)`,因為 `cascade=ALL`,Hibernate 會自動先刪 `review`、再刪 `course`。SQL 紀錄可以看到:

```

delete from course where id=10;
delete from review where course_id=10;

```

MySQL Workbench 重新整理後,課程與其評論都消失,驗證級聯刪除正確運作。

18. 多對多關聯 (@ManyToMany)

多對多可以想成「社團與社員」的關係:一個學生可以選很多門課,一門課也有很多個學生,雙方誰也不「擁有」誰。這種關係光靠兩張表互相加外鍵是做不到的,得靠一張中間的**連接表(Join Table)**當「報名表」,專門記錄「誰選了哪一門課」。

- **連接表的定義:**提供兩張表之間映射關係的表,裡面放著指向兩邊資料表的外鍵(FK),自己通常不需要額外的主鍵欄位。

- **範例結構**: `course_student` 連接表, 只有兩欄: `course_id`、`student_id`。

```
CREATE TABLE `course_student` (
  `course_id` int(11) NOT NULL,
  `student_id` int(11) NOT NULL,
  PRIMARY KEY (`course_id`, `student_id`),
  CONSTRAINT `FK_COURSE_05` FOREIGN KEY (`course_id`) REFERENCES `course`
(`id`),
  CONSTRAINT `FK_STUDENT` FOREIGN KEY (`student_id`) REFERENCES `student`
(`id`))
);
```

- **資料範例與查詢邏輯**(對照原文的「1. 資料表內容示範」「2. 如何透過連接表尋找關聯資料」):
 - 課程表: 10 號是 Pacman、11 號是 Rubik's Cube、12 號是 Atari 2600。學生表: 1 號 John、2 號 Mary。
 - 連接表紀錄: (10,2)、(11,2)、(12,2), 代表學生 2(Mary) 選了 10、11、12 三門課。
 - 想找「John 的課」: 先到 `student` 表查出 John 的 `id=1` → 到 `course_student` 找 `student_id=1` 的紀錄, 只找到一筆 `course_id=10` → 回 `course` 表查出課名是 Pacman。
 - 想找「Mary 的課」: 同樣先查出 `id=2` → 找到三筆紀錄(`course_id` 10、11、12) → 回 `course` 表查出三門課名。連接表就是這樣扮演「中間人」, 把原本互不相干的兩張表串起來。
- **Course 端映射預告**:

```
@ManyToMany
@JoinTable(
    name = "course_student",
    joinColumns = @JoinColumn(name = "course_id"),
    inverseJoinColumns = @JoinColumn(name = "student_id")
)
private List<Student> students;
```

- `joinColumns`: 指回「目前這個實體」(此例是 `Course`) 在連接表裡對應的欄位。
- `inverseJoinColumns`: 指向「關聯的另一方」(此例是 `Student`)。「Inverse」就是「關係的另一邊」的意思。
- **擁有方 / 反向方**: 雙向關聯可以自己選誰當擁有方, 但非擁有方一定要用 `mappedBy` 把關係「綁回」擁有端, 例如 `Student` 端寫 `@ManyToMany(mappedBy="students")`, 不需要也不該再寫一次 `@JoinTable`。

項目	擁有端 (Course)	反向端 (Student)
註解	@ManyToMany + @JoinTable	@ManyToMany(mappedBy="students")
職責	定義連接表實際結構	鏡射擁有端, 不定義物理結構

- **容易搞混的地雷**: 多對多不該設定級聯刪除 (`CascadeType.REMOVE` 或 `ALL`)。刪課程不該連帶砍掉學生, 刪學生也不該連帶砍掉課程——兩邊的生命週期要各自獨立, 這點跟一對多(刪課程要砍評論)剛好相反, 要特別留意別複製貼上錯設定。

19. 多對多關聯開發流程

- 開發三步驟:1. 準備工作(定義資料表,含連接表)→ 2. 更新 **Course** 類別 → 3. 更新 **Student** 類別。
- 建立連接表 **schema**:新開 **hb05-many-to-many** schema,執行腳本建立 **course_student**(複合主鍵 **course_id+student_id**,外鍵設 **ON DELETE NO ACTION ON UPDATE NO ACTION**,代表刪除父表資料不會自動連動處理連接表,要靠應用程式自己管)。
- 用 **Reverse Engineer** 畫 **EER** 圖:MySQL Workbench 選單 **Database → Reverse Engineer...**,選連線與目標 schema,勾選匯入資料表物件並放上圖表,就能自動產生視覺化的關聯圖,一眼看出 **course** 與 **student** 靠 **course_student** 多對多、**course** 與 **review** 一對多、**instructor** 與 **instructor_detail** 一對一。
- 專案準備:備份專案資料夾成 **05-jpa-many-to-many**,更新 **application.properties** 的 **spring.datasource.url** 指向 **hb-05-many-to-many**,清空 **CommandLineRunner** 裡的舊測試碼。
- 建立 **Student** 實體:對應 **student** 表(**id**、**first_name**、**last_name**、**email**):

```
@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "first_name")
    private String firstName;
    @Column(name = "last_name")
    private String lastName;
    @Column(name = "email")
    private String email;

    public Student() { }
    // 建構子只帶 firstName/lastName/email, id 不放進去(道理跟 Review 一樣,交給資料庫產生)
    // getter/setter、toString 用 IDE 自動生成
}
```

20. 更新 Course 實體類別以建立多對多關聯

這節把 **Course** 和 **Student** 兩端真正串起來,是多對多關聯的核心實作。

- **Course** 端:加入 **List<Student> students** 欄位,搭配便利方法防呆:

```
private List<Student> students;

public void addStudent(Student theStudent) {
    if (students == null) {
        students = new ArrayList<>();
    }
}
```

```
students.add(theStudent);
}
```

- 配置 **@ManyToMany + @JoinTable** (記得不要放 `CascadeType.REMOVE/ALL`, 只留存檔相關的級聯):

```
@ManyToMany(fetch = FetchType.LAZY,
             cascade = {CascadeType.PERSIST, CascadeType.MERGE,
                        CascadeType.DETACH, CascadeType.REFRESH})
@JoinTable(name="course_student",
            joinColumns = @JoinColumn(name="course_id"),
            inverseJoinColumns = @JoinColumn(name="student_id"))
private List<Student> students;
```

- **Student 端(反向端)**: 加入 `List<Course> courses`, 便利方法 `addCourse` 必須同時同步另一端, 這是雙向關聯最容易漏掉的地雷:

```
public void addCourse(Course theCourse) {
    if (courses == null) {
        courses = new ArrayList<Course>();
    }
    courses.add(theCourse);           // 更新自己這端
    theCourse.addStudent(this);       // 同步更新對方那端, 兩邊資料才會一致
}
```

Student 的關聯註解要用 `mappedBy` 指回 `Course` 那邊的欄位名稱, 而且不能再寫 `@JoinTable` (不然會多生出一張連接表):

```
@ManyToMany(fetch = FetchType.LAZY,
             cascade = {CascadeType.PERSIST, CascadeType.MERGE,
                        CascadeType.DETACH, CascadeType.REFRESH},
             mappedBy = "students")
private List<Course> courses;
```

- 測試: `createCourseAndStudents()` 建課程、建兩個學生、`tempCourse.addStudent()` 兩次、`appDAO.save(tempCourse)`。因為 `cascade` 有 `PERSIST`, 一次存檔會依序 insert `course`、insert `student` (兩筆)、最後 insert `course_student` (兩筆), 三張表一次搞定, 驗證了多對多的級聯儲存機制。

21. 透過學生查找其關聯課程

反過來, 從「學生」出發查他選了哪些課。

- **AppDAO** 新增方法與 JPQL 查詢, 一樣要用 `join fetch` 避免 N+1 問題 (逐筆額外查詢的效能地雷):

```

Student findStudentAndCoursesByStudentId(int theId);

// AppDAOImpl
TypedQuery<Student> query = entityManager.createQuery(
    "select s from Student s join fetch s.courses where s.id = :data",
    Student.class);
query.setParameter("data", theId);
Student student = query.getSingleResult();

```

- 更新學生的課程:AppDAO.update(Student) 用 entityManager.merge() (記得 @Transactional)。測試流程 addMoreCoursesForStudent():找出 Mary(id=2)→ 新建兩門課 (Rubik's Cube、Atari 2600)→ tempStudent.addCourse() 兩次(因為前面已經同步寫好雙向邏輯, Course 那端也會自動被更新)→ appDAO.update(tempStudent)。結果:新課程被存進 course 表拿到新 id, 連接表也新增對應紀錄, Mary 現在關聯到 10、11、12 三門課。
- 刪除課程, 只斷開關聯, 不動學生(呼應第 18 節「多對多不能亂用級聯刪除」的設計):deleteCourse 呼叫 deleteCourseById, Hibernate 會先清掉連接表裡對應的紀錄, 再刪課程本身:

```

delete from course_student where course_id=?;
delete from course where id=?;

```

學生資料完全不受影響, 只是少了跟這門課的關聯——這正是多對多刻意不設級聯刪除的效果。

- 刪除學生:必須手動解除雙向關聯, 因為沒有設 cascade REMOVE, 不能只靠 entityManager.remove(), 要先手動把學生從每一門課的學生名單裡移除, 不然會違反約束:

```

@Transactional
public void deleteStudentById(int theId) {
    Student tempStudent = entityManager.find(Student.class, theId);
    if (tempStudent != null) {
        List<Course> courses = tempStudent.getCourses();
        for (Course tempCourse : courses) {
            tempCourse.getStudents().remove(tempStudent); // 先斷開關聯
        }
        entityManager.remove(tempStudent); // 才能安全刪除
    }
}

```

- 章末複習重點:三種關聯的擁有方/反向方對照——@OneToOne(擁有方用 @JoinColumn, 反向方用 mappedBy)、@OneToMany(單向時 FK 在「多」那端且由該類別用 @OneToMany+@JoinColumn 管理; 雙向時搭配 @ManyToOne)、@ManyToMany(FK 都在連接表, 擁有端用 @JoinTable, 反向端用 mappedBy)。最後提醒:這些寫法是「通用指南」而非唯一正解, 實際專案要依業務需求彈性調整。

22. 面向切面程式設計 (AOP) 概觀

開頭情境:DAO 有一個 `addAccount()` 方法只做 `persist`。主管說要加 `log`,再來要加安全性檢查。如果每個方法都手動塞程式碼,會出現兩個典型問題:

- **程式碼糾結(Code Tangling)**:業務邏輯和 `log`、安全檢查全部黏在同一個方法裡,分不清哪段才是「正事」
- **程式碼分散(Code Scattering)**:同樣的 `log`/安全邏輯要複製到 `Controller`、`Service`、`DAO` 每一層、每一個類別,將來要改格式就要把所有類別重新翻一遍

試過的兩個替代方案都不夠好:

- **繼承**:做一個基礎類別把 `log`/安全性包起來,讓大家繼承——但 Java 只能單一繼承,而且舊類別還是得一個手動改成繼承它
- **委派**:把呼叫委派給 `LoggingManager`、`SecurityManager`——但要加新功能(審計、API 管理)時還是得回頭改每個類別,問題只是搬了位置,沒真的解決

這時候 AOP 登場。可以把 AOP 想成大樓的保全系統:不用在每個房間都裝一個警衛(改每個類別),而是在大樓入口設一個門禁系統(切面),規則設定好之後,所有房間自動套用同一套安全機制,完全不用動房間本身的裝潢(業務程式碼)。

核心術語:

- **Aspect(切面)**:處理橫切關注點(cross-cutting concerns,例如 `logging`、`security`)的模組,本質就是一個類別,可以重複套用到不同地方
- **Advice(通知)**:定義「要做什麼」以及「什麼時候做」
- **Join Point(關聯點)**:程式執行過程中可以插入程式碼的時間點
- **Pointcut(切點)**:一個斷言式,決定 Advice 要套用在哪些方法上

Advice 的五種類型:Before(方法執行前)、After returning(方法成功回傳後)、After throwing(方法拋出例外時)、After finally(方法結束後,不管成功失敗,像 `try-finally`)、Around(最強大,方法執行前後都能介入)。

運作機制是**代理模式(Proxy)**:Main App 呼叫的其實不是目標物件本身,而是一個 AOP Proxy。Proxy 先執行掛在上面的 Aspect(像先過安檢),再把呼叫轉給真正的 Target Object,整個過程對 Main App 完全透明,它根本不知道中間多了一層。**編織(Weaving)**就是把 Aspect 和 Target Object 綁在一起的過程,分編譯時、載入時、執行時三種——Spring AOP 用的是執行時編織(Run-time weaving),速度最慢,但換來簡單易用。

優點:邏輯集中好維護、業務程式碼保持乾淨、可配置(要套用在哪自己決定,不用動主程式)。缺點:切面太多會讓執行流程變得難追蹤(像太多間諜同時行動,搞不清楚誰先做了什麼),而且執行時編織有微小效能成本。常見應用:`logging`、`security`、`transactions`、`audit logging`(誰、做了什麼、何時、哪裡)、`exception handling`(自動通知 DevOps)、API 管理(呼叫次數、尖峰分析)。使用建議是「適量使用」,團隊要有治理規則,決定誰能建切面、切面能用在哪。

23. Java AOP 框架

Java 兩大 AOP 框架:**Spring AOP** 與 **AspectJ**。

Spring AOP:Spring 內建支援,框架本身的 `Security`、`Transactions`、`Caching` 都是靠 AOP 做的;用 Proxy Pattern + 執行時編織。優點是簡單好上手、用 `@Aspect` 註解、之後要換 AspectJ 也容易遷移;缺點是只支援方法層級的 Join Point、只能套用在 Spring App Context 建立的 Bean 上、有一點點效能成本。

AspectJ:2001 年就有的老牌框架,支援完整的 Join Point(方法、建構子、欄位層級都可以),編織時機更多元(編譯時、後編譯時、載入時)。優點是功能最完整、可以用在任何 POJO(不限 Spring Bean)、速度比 Spring AOP

快;缺點是編譯時編織要多一道編譯步驟、Pointcut 語法容易變得很複雜。

特性	Spring AOP	AspectJ
關聯點支援	只有方法層級	全部支援
對象限制	只能是 Spring Bean	任何 POJO
效能	較慢(執行時編織)	較快
複雜度	低	高

課程建議:先從 Spring AOP 開始,真的碰到 Spring AOP 搞不定的複雜需求,再考慮 AspectJ。

Spring Boot 專案要用 AOP,只要在 `pom.xml` 加上:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

依賴一加進去, Spring Boot 會自動幫你啟用 AOP(不像傳統專案還要手動加 `@EnableAspectJAutoProxy`, Spring Boot 直接「免費送」)。

實作三步驟示範(以 `@Before` 為例):① 建立目標物件(Target Object),例如 `AccountDAO` 介面 + `AccountDAOImpl` 實作,記得加 `@Component` 讓 Spring 管理;② 建立主應用程式,用 `CommandLineRunner` 在啟動後呼叫業務方法;③ 建立切面,加上 `@Aspect` + `@Component` 兩個註解——`@Aspect` 告訴 Spring「這是切面」, `@Component` 讓它被組件掃描抓到,兩個缺一不可:

```
@Aspect
@Component
public class MyDemoLoggingAspect {

    @Before("execution(public void addAccount())")
    public void beforeAddAccountAdvice() {
        System.out.println("\n====>> Executing @Before advice on
addAccount();");
    }
}
```

`execution(public void addAccount())` 就是 Pointcut 表達式:意思是「只要有 public、回傳 void、叫 `addAccount` 的方法被呼叫,就先執行這段通知」。

最後有個重要開發心法:Advice 裡的程式碼要「進去就快點出來(get in and out as quickly as possible)」——保持輕量、避免昂貴操作,因為它會插在每一次符合條件的方法呼叫上,拖慢一點點就是全面拖慢。

24. AOP 示範專案初始化

用 Spring Initializr(start.spring.io)建新專案,設定重點:Project 選 Maven、Language 選 Java、Spring Boot 版本選最新正式版(不要選 SNAPSHOT,那是還在測試的 beta);Metadata 例如 Group=com.love2code、Artifact=aopdemo、Package=com.love2code.aopdemo、Packaging=Jar。

要注意 Spring Initializr 網頁上不一定找得到 AOP 這個勾選項,下載完專案後要自己手動去 pom.xml 補上:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

加完記得 Reload/Sync Maven,不然 IDE 抓不到新依賴。專案整理習慣:解壓縮後搬到開發資料夾,資料夾重新命名成有意義的名字(例如 01-spring-boot-aop-demo),方便之後同一主題有多個版本時互相區分。

application.properties 順手做清潔工作,讓 console 輸出乾淨好讀:

```
# Reduce logging level. Set logging level to warn
logging.level.root=warn
```

再加上關掉開機 banner。接著搭骨架:先用 CommandLineRunner 印一句 "Hello world!" 確認專案能跑,再依照三步驟開發 AOP:① 建立目標物件 AccountDAO(介面 + Impl,Impl 加 @Repository,它其實是 @Component 的子類型);② 主程式用 CommandLineRunner 呼叫 accountDAO.addAccount() (此時尚未接 AOP,先確認業務邏輯本身沒問題,輸出應該是 "DOING MY DB WORK: ADDING AN ACCOUNT");③ 建立切面套件 com.love2code.aopdemo.aspect,裡面放 MyDemoLoggingAspect,加 @Aspect + @Component。

寫好 @Before advice 後執行,console 應該先看到切面訊息、才看到 DAO 訊息:

```
====>> Executing @Before advice on addAccount();
class com.love2code.aopdemo.dao.AccountDAOImpl: DOING MY DB WORK: ADDING AN
ACCOUNT
```

這一節重點其實不是新知識,而是把上一節的三步驟真的手刻一遍,確認流程能跑通,為下一節深入 Pointcut 語法打地基。

25. 點切點表達式 (Pointcut Expressions)

Pointcut 是一個斷言式(predicate),Spring AOP 用的是 AspectJ 的表達式語言,最常用的是 execution,專門匹配「方法的執行」。完整語法長這樣:

```
execution(modifiers-pattern? return-type-pattern declaring-type-pattern?
method-name-pattern(param-pattern) throws-pattern?)
```

帶問號的都是選用:修飾符、宣告類型(類別路徑)、throws 都可以省略,實務上最常寫的其實就是「回傳類型 + 方法名稱(參數)」這幾個核心欄位。

從精確到寬鬆,幾種常見寫法比較:

```
// 完全限定類別路徑,只匹配 AccountDAO 這個類別的 addAccount
@Before("execution(public void
com.luv2code.aopdemo.dao.AccountDAO.addAccount())")

// 不寫類別路徑,任何類別的 addAccount() 都會中
@Before("execution(public void addAccount())")

// 用 * 當萬用字元,任何以 add 開頭的方法都會中
@Before("execution(public void add*())")

// 連修飾符都省略,範圍最寬
@Before("execution(* add*())")
```

這裡有個很實用也很容易踩雷的觀念:表達式寫得越寬,攔截範圍就越大,可能連你不想攔截的方法都一起中招。書裡實際做了個實驗——先在 `AccountDAO` 攔截 `addAccount()`,故意把 Pointcut 改成 `updateAccount()` (這個方法根本不存在),結果 Advice 完全不會被觸發,console 甚至提示「never called because no calls to: updateAccount()」。這證明 Pointcut 匹配的是「方法簽章」,不是「你以為它會匹配的東西」,寫錯字或改錯範圍,Advice 就會靜悄悄地失效或誤觸發,每次修改後都值得實際跑一次驗證。

另一個實驗是複製一個新的 `MembershipDAO`,也做一個 `addAccount()` 方法。因為 Pointcut 沒寫死類別路徑,結果兩個類別的 `addAccount()` 都被攔截了——說明「省略宣告類型」等於把攔截範圍打開給所有類別,用起來要很小心,不然會誤傷不相關的類別。想收斂範圍,加回完全限定類別名稱(Fully Qualified Class Name)即可,例如 `execution(public void com.luv2code.aopdemo.dao.AccountDAO.addAccount())` 就只會匹配 `AccountDAO`,不會匹配 `MembershipDAO`。

小結一句話:`execution` 表達式的每個欄位都可以省略或用 `*` 放寬,越寬泛越通用但風險也越高;越精確越安全但要多寫字,實務上要在「好用」和「精準」之間抓平衡。

26. 根據回傳類型進行方法匹配

前一節主要在調方法名稱跟類別路徑,這節焦點換成 `execution` 表達式裡的「回傳類型」欄位。

先做個對照實驗:把點切點從 `execution(public void add*())` 改成 `execution(void add*())` (拿掉 `public`),意思是「不管存取修飾符是什麼,只要回傳 `void` 且方法名以 `add` 開頭就攔截」。接著故意把 `MembershipDAO` 的 `addSillyMember()` 回傳型別從 `void` 改成 `boolean`——結果這個方法不再被攔截,因為回傳型別跟表達式要求的 `void` 對不上。這證明回傳型別是真正會影響匹配結果的過濾條件,不是裝飾用的。想讓表達式同時吃 `void` 和 `boolean` 等各種回傳型別,回傳型別位置也可以用萬用字元 `*`:`@Before("execution(* add*())")`,這樣不管方法回傳什麼,只要名字以 `add` 開頭都會中。

這節也把「參數模式(Parameter Pattern)」的三種寫法講清楚,是接下來組合 Pointcut 的基礎:

參數模式	匹配描述
<code>()</code>	只匹配「無參數」的方法
<code>(com.package.ClassName)</code>	只匹配「帶特定型別參數」的方法
<code>(..)</code>	匹配零個或多個、任何型別的參數(最寬鬆)

書裡示範了完整流程:先幫 `addAccount()` 加一個 `Account` 參數,Pointcut 用 `execution(* addAccount(com.luv2code.aopdemo.Account))` 精確匹配;後來需求變了,方法又多一個 `boolean vipFlag` 參數,原本寫死單一參數型別的表達式立刻失效(console 顯示「No match」)——這是另一個常見陷阱:方法簽章一改,寫死參數型別的 Pointcut 就會跟著壞掉,而且不會報錯,只是安靜地不再攔截。解法是用 `..` 收尾,例如 `execution(* add*(com.luv2code.aopdemo.Account, ..))`,代表「第一個參數是 `Account`,後面隨便你加幾個都行」;如果連第一個參數型別都不想管,直接寫 `execution(* add*(..))` 最通用。

最後這節也提醒一個實務風險:Pointcut 用萬用字元用得太多,有可能誤中專案以外、不相干的框架類別。書裡舉例 IntelliJ Ultimate 會多載入一些 JMX 相關類別,如果 Pointcut 沒有收斂在自己的套件範圍內,可能會跟 Spring Boot 的 `JmxAutoConfiguration` 打架,丟出 `BeanCreationException`。建議做法是把 Pointcut 收斂在自己的專案套件下:

```
@Before("execution(* com.luv2code.aopdemo.dao.*.*(..))")
```

`dao..` 代表「dao 套件及其所有子套件」,`*.*` 代表「任何類別的任何方法」,`(..)` 代表任何參數——組合起來就是「只要是 dao 套件底下的東西,不管哪個類別哪個方法,通通攔截」。測試時新增的 `doWork()`、`goToSleep()` 等各種不同名稱的方法也都能被成功攔到,驗證了這種「鎖套件、不鎖方法名」的寫法適合用在整層(例如整個 DAO 層)都要套用同一種橫切邏輯的情境。

27. 點切點宣告 (Pointcut Declarations)

如果一個切面裡有好幾個 `Advice(@Before、@After...)` 都要攔截同一批方法,一個個把 `execution(...)` 表達式複製貼上進去,是最偷懶但也最不理想的作法——之後要改攔截範圍,就得把每一處複製貼上的地方都找出來改一遍,非常容易漏改。

解法是把 Pointcut「宣告」成一個獨立、有名字的東西,之後所有 `Advice` 都用這個名字去引用它,概念上有點像把重複用到的條件寫成一個變數。做法用 `@Pointcut` 註解 + 一個空方法:

```
@Pointcut("execution(* com.luv2code.aopdemo.dao.*.*(..))")
private void forDaoPackage() {}
```

`forDaoPackage` 不是一個真的會被執行的方法,它只是拿來「掛」`@Pointcut` 表達式、給這條規則取個名字用的容器。定義好之後,`Advice` 就可以直接引用這個名字,取代原本落落的 `execution(...)`:

```
@Aspect
@Component
public class MyDemoLoggingAspect {

    @Pointcut("execution(* com.luv2code.aopdemo.dao.*.*(..))")
    private void forDaoPackage() {}

    @Before("forDaoPackage()")
    public void beforeAddAccountAdvice() {
        System.out.println("\n---> Executing @Before advice on method");
    }
}
```

```

@Before("forDaoPackage()")
public void performApiAnalytics() {
    System.out.println("\n---> Performing API analytics");
}
}

```

好處很直接:往後只要改 `@Pointcut` 那一行,兩個 Advice 的攔截範圍就同步更新,不用到處找取代。

更進一步,Pointcut 宣告之間還可以用邏輯運算子組合,寫法跟寫 if 判斷式一模一樣:`&&`(AND,兩個條件都要成立)、`||`(OR,任一條件成立就算數)、`!`(NOT,排除某個條件)。一個很實用的例子是「攔截某個套件下所有方法,但排除 getter/setter」。先分別定義三個 Pointcut:

```

@Pointcut("execution(* com.luv2code.aopdemo.dao.*.*(..))")
private void forDaoPackage() {}

@Pointcut("execution(* com.luv2code.aopdemo.dao.*.get*(..))")
private void getter() {}

@Pointcut("execution(* com.luv2code.aopdemo.dao.*.set*(..))")
private void setter() {}

```

再組合成一個「dao 套件、但不含 getter 和 setter」的規則:

```

@Pointcut("forDaoPackage() && !(getter() || setter())")
private void forDaoPackageNoGetterSetter() {}

```

拆解邏輯:`(getter() || setter())` 先找出所有 getter 或 setter,前面加 `!` 表示「排除這些」,再用 `&&` `forDaoPackage()` 限定範圍只在 dao 套件內。最後把組合好的規則套到 Advice 上:

```

@Before("forDaoPackageNoGetterSetter()")
public void beforeAddAccountAdvice() {
    // ...
}

```

書裡也提醒了一個容易忽略的驗證步驟:在真正排除之前,要先故意讓「還沒排除」的版本跑一次,確認 getter/setter 真的有被目前寬鬆的 Pointcut(`forDaoPackage()`)攔截到,才能對照出「排除」生效前後的差異——如果一開始就直接寫組合表達式,很難判斷排除邏輯到底有沒有真的起作用,還是本來就沒攔到。這是驗證 AOP 規則時一個蠻實用的習慣:先看到「有攔到」,再看到「排除後沒攔到」,兩段對照才算完整驗證。

28. 存取與顯示方法參數

前面的 `@Before` advice 只會印方法名稱,這節要讓它「偷看」傳進去的參數是什麼。關鍵就一個方法:`JoinPoint.getArgs()`,它會把該次呼叫傳入的所有參數包成一個 `Object[]` 陣列丟給你,再用 for-each 迴圈把每個參數印出來。


```

@Before("com.luv2code.aopdemo.aspect.LuvAopExpressions.forDaoPackageNoGetterSetter()")
public void beforeAddAccountAdvice(JoinPoint theJoinPoint) {
    MethodSignature methodSignature = (MethodSignature)
theJoinPoint.getSignature();
    System.out.println("Method: " + methodSignature);

    Object[] args = theJoinPoint.getArgs();
    for (Object tempArg : args) {
        System.out.println(tempArg);
    }
}

```

問題來了:如果參數是自訂物件(例如 `Account`)又沒 override `toString()`,印出來的只會是一串沒意義的 hash code。解法是用 `instanceof` 判斷型別、再做 downcast,這樣才能呼叫 `getName()`、`getLevel()` 這類專屬方法:

```

for (Object tempArg : args) {
    if (tempArg instanceof Account) {
        Account theAccount = (Account) tempArg;
        System.out.println("account name: " + theAccount.getName());
        System.out.println("account level: " + theAccount.getLevel());
    }
}

```

這裡有個容易誤判成 bug 的陷阱:第一次測試時如果 console 印出 `account name: null`,不代表程式邏輯寫錯,通常是因為主程式建立的 `Account` 物件根本沒設定任何屬性(空殼物件)。解法很單純,回到呼叫端 (`AopdemoApplication`)手動 `setName()`、`setLevel()` 給測試資料填值,重跑一次就會看到正確內容。這一節其實是在提醒:AOP 邏輯沒問題時,先檢查測試資料是不是準備齊全,別急著懷疑 Aspect。

到這裡,`@Before` advice、`Pointcut` 表達式、`JoinPoint` 這三塊算是打底完成,接下來要進入其他 advice 類型:`@AfterReturning`、`@AfterThrowing`、`@After`、`@Around`。

29. `@AfterReturning` Advice - 修改回傳值

`@AfterReturning` 除了拿來寫 log、做安全檢查之外,還有一招更猛的用法:在資料回到呼叫者手上之前,直接把它改掉(格式化或加料都算)。做法是靠 `returning` 屬性把方法回傳值綁定到一個參數上,拿到的就是原始的 `List<Account>` 物件參考,你對它做任何增刪改,呼叫端拿到的就是「已經被動過手腳」的版本:

```

@AfterReturning(
    pointcut = "execution(*
com.luv2code.aopdemo.dao.AccountDAO.findAccounts(...))",
    returning = "result"
)
public void afterReturningFindAccountsAdvice(JoinPoint theJoinPoint,
List<Account> result) {
    for (Account account : result) {

```



```

        account.setName(account.getName().toUpperCase());
    }
}

```

程式跑起來後,主程式呼叫 `findAccounts()` 拿到的名字全部變大寫,但主程式的程式碼一行都沒改——它完全不知道背後有人動過資料。

這節花不少篇幅在講「這功能雖強但要小心用」的道理,用了一個很生活化的比喻:就像你網購的包裹在送達前被某個單位攔截拆封、動了手腳,可能是好事(加了贈品)也可能是壞事(被調包)。如果團隊裡沒人知道系統裡藏著這種 AOP 攔截器,遇到「明明查詢邏輯沒改,資料卻怪怪的」這種狀況會抓破頭都查不出原因。所以這裡的重點與其說是技術,不如說是工程紀律:團隊必須清楚知道專案裡有哪些 **Aspect** 在背後動資料,不然遲早會出現難以追蹤的靈異 bug。

30. @AfterThrowing Advice

跟 `@AfterReturning` 恰好相反的情境:目標方法拋出例外時才會觸發。幾種 advice 放一起比較會更清楚:

Advice	觸發時機
<code>@Before</code>	方法執行前
<code>@AfterReturning</code>	方法「成功」執行完(無例外)後
<code>@AfterThrowing</code>	方法拋出例外時
<code>@After(finally)</code>	不管成功或失敗,方法結束後一定執行
<code>@Around</code>	方法執行前後都能介入,控制力最強

要拿到例外物件本身,用法跟 `@AfterReturning` 的 `returning` 幾乎一樣,只是換成 `throwing` 屬性,對應到一個 `Throwable` 型別的參數(名稱要完全一致):

```

@AfterThrowing(
    pointcut = "execution(*
com.luv2code.aopdemo.dao.AccountDAO.findAccounts(..))",
    throwing = "theExc"
)
public void afterThrowingFindAccountsAdvice(JoinPoint theJoinPoint,
Throwable theExc) {
    System.out.println("\n>>> The exception is: " + theExc);
}

```

最重要的觀念在這:`@AfterThrowing` 只是讓你「窺視」一下例外、順便記個 log,它擋不住例外繼續往上傳——異常還是會原封不動地一路飛回呼叫程式,該 `catch` 還是得 `catch`。如果目標是真的把例外「吞掉」讓呼叫者感覺不到,`@AfterThrowing` 做不到,得換 `@Around` 才有這種控制力。常見用途是記錄例外、做稽核軌跡、或通知團隊(但要挑「真的很嚴重」的錯誤才通知,不然會變成 狼來了式的訊息轟炸)。

實作測試上用了一個很典型的手法:在 DAO 方法上加一個 `boolean tripWire` 參數,`tripWire=true` 就手動 `throw new RuntimeException(...)` 模擬異常,`tripWire=false` 走正常路徑,這樣可以用同一支方法同時測「失敗案例」跟「成功案例」。

這節後段也帶到了 `@After` (也叫 `after-finally`): 行為完全對應 Java 的 `finally` 區塊, 不管方法成功還是丟例外, 都保證會執行。要注意的是它拿不到例外物件 (要拿例外還是得用 `@AfterThrowing`), 所以寫在裡面的邏輯不該假設走的是哪條路徑, 最適合放日誌、稽核這種不需要分支判斷的通用工作。

31. @Around Advice

`@Around` 是威力最大的 advice, 可以想成 `@Before` + `@After` 的合體, 而且控制粒度更細——它能決定目標方法到底要不要執行、執行前後都能插邏輯、還能攔截並改寫例外。核心工具是 `ProceedingJoinPoint`, 它是目標方法的一個「遙控器」(handle), 呼叫它的 `proceed()` 才會真的觸發目標方法執行:

```
@Around("execution(* com.luv2code.aopdemo.service.*.getFortune(..)")
public Object aroundGetFortune(ProceedingJoinPoint theProceedingJoinPoint)
throws Throwable {

    String method = theProceedingJoinPoint.getSignature().toShortString();
    System.out.println("\n---> Executing @Around on method: " + method);

    long begin = System.currentTimeMillis();
    Object result = theProceedingJoinPoint.proceed();    // 真正執行目標方法
    long end = System.currentTimeMillis();

    long duration = end - begin;
    System.out.println("\n---> Duration: " + duration / 1000.0 + "
seconds");

    return result;    // 一定要回傳, 不然呼叫端拿不到資料
}
```

這裡有兩個新手最容易踩的坑, 務必記牢:

- 忘記呼叫 `proceed()`: 目標方法會完全不執行, 呼叫端什麼都拿不到, 卻不會報錯, 很難察覺。
- 忘記 `return result`: 即使 `proceed()` 有執行, 呼叫端收到的還是空的, 因為你沒把結果傳回去。

範例用 `TrafficFortuneService.getFortune()` 搭配 `TimeUnit.SECONDS.sleep(5)` 模擬延遲, 拿 `@Around` 做效能分析 (instrumentation/profiling): 執行前後各記一個時間戳、相減算耗時。後面會發現一個小坑——如果方法很快就拋出例外, 用 `System.currentTimeMillis()` (毫秒) 算出來的 `duration` 可能顯示 `0.0 seconds`, 這不是沒執行, 是精度不夠; 換成 `System.nanoTime()` (奈秒) 就能量到真實的微小耗時。

`@Around` 的應用場景比其他 advice 都廣: 日誌、稽核、安全性、資料前處理/後處理、效能分析, 以及下一節要講的例外管理 (能吞、能處理、能擋, 不像 `@AfterThrowing` 只能看)。

32. @Around Advice - 異常處理

延續上一節, `@Around` 對例外有真正的控制權, 做法就是用 `try-catch` 包住 `proceed()`:

```
Object result = null;
try {
    result = theProceedingJoinPoint.proceed();
} catch (Exception exc) {
```

```

        System.out.println("@Around advice: We have a problem " + exc);
        result = "Nothing exciting here. Move along!";    // 給預設值,吞掉例外
    }
    return result;

```

這裡分出兩種策略,用一個上班族都懂的比喻來記最好記:

- **方案 A:自己處理、給預設值(吞掉例外)**——像遇到小事自己扛,不驚動主管。呼叫程式完全不會知道曾經出過錯,程式能繼續跑下去,但風險是把真正的問題藏起來了。
- **方案 B:log 完之後 `throw exc`; 重新拋出**——像遇到大事要立刻通報主管。呼叫端一樣會收到例外、該怎麼處理由它自己決定,只是 advice 幫忙留了一筆記錄。

```

catch (Exception exc) {
    System.out.println("@Around advice: We have a problem " + exc);
    throw exc;    // 記錄後照樣往上丟
}

```

課程特別提醒一個工程紀律問題:不要把「吞例外給預設值」當 OK 繃亂貼。如果某段程式碼每次呼叫都在丟例外,那是 code smell,代表根源沒解決;比較健康的做法是去找出源頭 bug 把它修好,而不是靠 AOP 層長期「假裝沒事」。方案 A 只適合真正無傷大雅的小狀況。

跟上一節一樣的精度陷阱在這裡也會出現:例外路徑下用 `currentTimeMillis()` 算出來的 duration 常常是 `0.0 seconds`,換成 `System.nanoTime()` 才能看到真實數字(單位跟著改成奈秒,不用再除以 1000.0)。

33. AOP 與 Spring MVC 的整合

前面都是在玩具專案裡練功,這節要把整套 AOP 技巧套進一個真實的 Spring MVC CRUD 專案(Employee Directory,含 Controller → Service → DAO → 資料庫的完整分層),目標是讓每個請求進出各層時自動被記錄。

開發步驟大致是:

1. `pom.xml` 加 `spring-boot-starter-aop` 依賴(Spring Boot 偵測到就會自動啟用 AOP,不用額外設定)。

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>

```

2. 建立 `aspect` 套件與 `DemoLoggingAspect` 類別,標上 `@Aspect` + `@Component` (讓 Spring 組件掃描能發現它),並用 `java.util.logging.Logger` (而不是 `System.out.println`) 來輸出:

```

@Aspect
@Component
public class DemoLoggingAspect {

```

```
private Logger myLogger = Logger.getLogger(getClass().getName());
}
```

3. 設定 **Pointcut**:分別針對 `controller`、`service`、`dao` 三個套件各自定義一個切點,再用 `||` 把它們組合成一個總開關,故意排除 `entity` 套件避免攔截到不相關的類別:

```
@Pointcut("execution(* com.luv2code.springboot.thymeleafdemo.controller.*.*(..))")
private void forControllerPackage() {}

@Pointcut("execution(* com.luv2code.springboot.thymeleafdemo.service.*.*(..))")
private void forServicePackage() {}

@Pointcut("execution(* com.luv2code.springboot.thymeleafdemo.dao.*.*(..))")
private void forDaoPackage() {}

@Pointcut("forControllerPackage() || forServicePackage() || forDaoPackage()")
private void forAppFlow() {}
```

4. **@Before advice**:套用組合切點 `forAppFlow()`,印出方法名稱與所有參數(還是靠 `getSignature().toShortString()` 和 `getArgs()` 這套老班底):

```
@Before("forAppFlow()")
public void before(JoinPoint theJoinPoint) {
    String theMethod = theJoinPoint.getSignature().toShortString();
    myLogger.info("in @Before: calling method: " + theMethod);

    for (Object tempArg : theJoinPoint.getArgs()) {
        myLogger.info("in @Before: argument: " + tempArg);
    }
}
```

5. **@AfterReturning advice**:同樣掛在 `forAppFlow()` 上,用 `returning` 抓每一層回傳的資料:

```
@AfterReturning(pointcut = "forAppFlow()", returning = "theResult")
public void afterReturning(JoinPoint theJoinPoint, Object theResult) {
    myLogger.info("##### in @AfterReturning: from method: " + theJoinPoint.getSignature());
    myLogger.info("##### result: " + theResult);
}
```

實際跑起來(瀏覽器操作「更新員工」「新增員工」等流程)後,console 會清楚看到一條完整的呼叫鏈:請求先進 `EmployeeController`,再往下傳到 `EmployeeServiceImpl`,最後到 `CrudRepository`(DAO 層),每一層的

方法名稱與傳入參數(甚至是完整的 `Employee` 物件內容)都被精準記錄下來;回傳路徑也是一樣,資料從 DAO 往上回傳到 Service、Controller 時同樣被 `@AfterReturning` 捕捉。這就驗證了 AOP 不只是能攔截單一方法呼叫,而是能貫穿整個分層架構,把日誌邏輯統一抽出來、不用在每個 Controller/Service/DAO 裡手動加 log 陳述式,這正是 AOP 「橫切關注點」的價值所在。